08장 정규표현식

필자는 '정규 표현식'을 이 책에 다뤄야 할지 오랫동안 고민했다. 정규 표현식은 꽤 오랜 기간 코드를 작성해 온 개발자라도 잘 모를 수 있는 고급 주제여서 초보자를 대상으로 하는 이 책에는 어울리지 않을 수 있기 때문이다.

하지만 정규 표현식을 익히기만 하면 매우 달콤한 열매를 맛볼 수 있다. 그래서 파이썬 공식 문서에서 소개하는 내용(docs.python.org/3.14/howto/regex.html)을 참고해 그곳에서 소개하는 수준의 내용만이라도 독자들이 이해하고 사용할 수 있도록 08장을 집필했다. 정규 표현식을 잘 다루게 되면 여러분은 파이썬 말고도 또 하나의 강력한 무기를 얻게 될 것이다.

다시 한번 말하지만 프로그래밍 입문자가 이해하기에는 어려운 내용이므로 부담갖지 말고 편하게 산책하듯 읽어 보기 바란다.

목차

- 08-1 정규 표현식 살펴보기
- 08-2 정규 표현식 시작하기
- 08-3 강력한 정규 표현식의 세계로

08-1 정규 표현식 살펴보기

정규 표현식(regular expressions)은 **문자열의 패턴을 표현하는 특별한 문법**이다.

쉽게 말해, 정규 표현식은 "이런 형태의 문자열을 찾고 싶다"라고 컴퓨터에게 알려주는 방법이다. 예를 들어 "숫자 6개 다음에 하이픈(-), 그리고 숫자 7개로 이루어진 문자열"이라고 설명하는 대신, 정규 표현식으로는 `\d{6}-\d{7}`처럼 간단하게 표현할 수 있다. 이렇게 하면 주민등록번호 같은 특정 패턴의 문자열을 쉽게 찾거나 검증할 수 있다.

정규 표현식은 파이썬만의 고유 문법이 아니라 자바스크립트, 자바, C# 등 거의 모든 프로그래밍 언어에서 사용하는 표준적인 문자열 처리 도구이다. 한 번 배워두면 어떤 언어를 사용하든 문자열 처리가 훨씬 쉬워진다.

> *정규 표현식은 줄여서 '정규식'이라고도 한다.*

정규 표현식은 왜 필요한가?

다음과 같은 문제가 주어졌다고 가정해 보자.

```
주민등록번호를 포함하고 있는 텍스트가 있다. 이 텍스트에 포함된 모든 주민등록번호의 뒷자리를 * 문자로 변경해 보자.
```

정규식을 모르면 다음과 같은 순서로 프로그램을 작성해야 한다.

1. 전체 텍스트를 공백 문자로 나눈다(split).
2. 나뉜 단어가 주민등록번호 형식인지 조사한다.
3. 단어가 주민등록번호 형식이라면 뒷자리를 `*`로 변환한다.
4. 나뉜 단어를 다시 조립한다.

이를 구현한 코드는 다음과 같다.

```
data = """
park 800905-1049118
kim  700905-1059119
"""

result = []
for line in data.split("\n"):
    word_result = []
    for word in line.split(" "):
        if len(word) == 14 and word[:6].isdigit() and word[7:].isdigit():
            word = word[:6] + "-" + "*******"
        word_result.append(word)
    result.append(" ".join(word_result))
print("\n".join(result))
```

[실행 결과]

```
park 800905-*******
kim  700905-*******
```

반면, 정규식을 사용하면 훨씬 간편하고 직관적인 코드를 작성할 수 있다. 아직 정규식 사용 방법을 배우지 않았으니 일단 어떤 모습인지만 살펴보자.

```
import re

data = """
park 800905-1049118
kim  700905-1059119
"""

pat = re.compile(r"(\d{6})-\d{7}")
print(pat.sub("\g<1>-*******", data))
```

[실행 결과]

```
park 800905-*******
kim  700905-*******
```

정규 표현식을 사용하면 코드가 상당히 간결해진다. 찾으려는 문자열 또는 바꾸어야 할 문자열의 규칙이 복잡할수록 정규식의 효용은 더 커진다.

이제부터 정규 표현식의 기초부터 심화 부분까지 차근차근 알아보자.

08-2 정규 표현식 시작하기

정규 표현식에서는 메타 문자(meta characters)를 사용한다. 먼저 메타 문자가 무엇인지 알아보자.

> *이 장에서는 "매치(match)"라는 용어가 자주 등장한다. 매치란 정규식 패턴과 문자열이 일치하는 것을 말한다. 예를 들어 정규식 `abc` 에 문자열 "abc"가 일치하면 "매치된다"라고 표현한다.*

- 정규 표현식의 기초, 메타 문자
  - [ ] 문자 - 문자 클래스
  - .(dot) 문자 - \n을 제외한 모든 문자
  - * 문자
  - + 문자
  - {} 문자와 ? 문자
- 파이썬에서 정규 표현식을 지원하는 re 모듈
- 정규식을 이용한 문자열 검색
  - match
  - search
  - findall
  - finditer
- match 객체의 메서드
- 컴파일 옵션
  - DOTALL, S
  - IGNORECASE, I
  - MULTILINE, M
  - VERBOSE, X
- 역슬래시 문제

정규 표현식의 기초, 메타 문자

메타 문자란 원래 그 문자가 가진 뜻이 아니라 특별한 의미를 가진 문자를 말한다. 정규 표현식에 다음과 같은 메타 문자를 사용하면 특별한 의미를 갖게 된다.

```
. ^ $ * + ? { } [ ] \ | ( )
```

그러면 가장 간단한 정규 표현식부터 시작해 각 메타 문자의 의미와 사용법을 알아보자.

[ ] 문자 - 문자 클래스

우리가 가장 먼저 살펴볼 메타 문자는 바로 문자 클래스(character class)인 `[]` 이다.

문자 클래스는 "이 중에서 아무거나 하나"라는 의미를 가진다. 예를 들어 `[abc]` 라고 쓰면 "a 또는 b 또는 c 중에서 문자 하나"를 의미한다. 마치 여러 개의 선택지 중에서 하나를 고르는 것과 같다.

> *문자 클래스를 만드는 메타 문자인 [ ] 사이에는 어떤 문자도 들어갈 수 있다.*

좀 더 구체적으로 정규 표현식 `[abc]` 의 의미를 알아보자:

- "a, b, c 중 정확히 한 개의 문자가 있으면 매치"
- 주의할 점은 abc 전체가 아니라 그중 하나의 문자만 찾는다는 것이다

실제 예시로 문자열 "a", "before", "dude"가 정규식 `[abc]` 와 어떻게 매치되는지 살펴보자:

- "a" → 매치됨 (문자 "a"가 [abc] 안에 포함되어 있음)
- "before" → 매치됨 (문자 "b"가 [abc] 안에 포함되어 있음)
- "dude" → 매치되지 않음 (d, u, d, e 모두 [abc] 안에 없음)

범위 지정하기 - 하이픈(-) 사용법

[] 안의 두 문자 사이에 하이픈(-)을 사용하면 두 문자 사이의 범위를 간편하게 표현할 수 있다. 연속된 문자들을 일일이 나열하지 않아도 된다.

예시:

- `[a-c]` → `[abc]` 와 동일 (a부터 c까지)
- `[0-5]` → `[012345]` 와 동일 (0부터 5까지)
- `[A-Z]` → 모든 대문자 알파벳 (A부터 Z까지)

자주 사용하는 범위 표현:

- `[a-zA-Z]` : 모든 알파벳 (소문자와 대문자 모두)
- `[0-9]` : 모든 숫자 (0부터 9까지)
- `[가-힣]` : 모든 한글 (가부터 힣까지)

제외하기 - `^` 사용법

문자 클래스 맨 앞에 `^`를 사용하면 "~가 아닌"이라는 의미가 된다. 즉, 해당 문자들을 제외한 모든 문자와 매치된다.

예시:

- `[^0-9]` → 숫자가 아닌 모든 문자 (알파벳, 특수문자, 공백 등)
- `[^abc]` → a, b, c가 아닌 모든 문자
- `[^A-Z]` → 대문자가 아닌 모든 문자

점프 투 파이썬

자주 사용하는 문자 클래스

`[0-9]` 또는 `[a-zA-Z]` 등은 무척 자주 사용하는 정규 표현식이다. 이렇게 자주 사용하는 정규식은 별도의 표기법으로 표현할 수 있다. 다음을 기억해 두자.

- `\d` - 숫자와 매치된다. `[0-9]`와 동일한 표현식이다.
- `\D` - 숫자가 아닌 것과 매치된다. `[^0-9]`와 동일한 표현식이다.
- `\s` - 화이트스페이스(whitespace) 문자와 매치된다. `[ \t\n\r\f\v]`와 동일한 표현식이다. 맨 앞의 빈칸은 공백 문자(space)를 의미한다.
- `\S` - 화이트스페이스 문자가 아닌 것과 매치된다. `[^ \t\n\r\f\v]`와 동일한 표현식이다.
- `\w` - 문자+숫자(alphanumeric)와 매치된다. `[a-zA-Z0-9_]`와 동일한 표현식이다.
- `\W` - 문자+숫자(alphanumeric)가 아닌 문자와 매치된다. `[^a-zA-Z0-9_]`와 동일한 표현식이다.

대문자로 사용된 것은 소문자의 반대임을 추측할 수 있다.

.(dot) 문자 - `\n`을 제외한 모든 문자

정규 표현식의 .(dot) 메타 문자는 줄바꿈 문자인 `\n`을 제외한 모든 문자와 매치된다는 것을 의미한다.

> *정규식을 작성할 때 `re.DOTALL` 옵션을 주면 .(dot) 문자와 `\n` 문자도 매치된다.*

다음 정규식을 살펴보자.

```
a.b
```

위 정규식의 의미는 다음과 같다.

```
"a + 모든_문자 + b"
```

즉, a와 b라는 문자 사이에 어떤 문자가 들어가도 모두 매치된다는 의미이다.

이해를 돕기 위해 문자열 "aab", "a0b", "abc"가 정규식 `a.b`와 어떻게 매치되는지 살펴보자.

- "aab"는 가운데 문자 "a"가 모든 문자를 의미하는 `.`과 일치하므로 정규식과 매치된다.
- "a0b"는 가운데 문자 "0"이 모든 문자를 의미하는 `.`과 일치하므로 정규식과 매치된다.
- "abc"는 "a" 문자와 "b" 문자 사이에 어떤 문자라도 하나는 있어야 하는 이 정규식과 일치하지 않으므로 매치되지 않는다.

앞의 식과 조금 다른 다음 정규식을 살펴보자.

```
a[.]b
```

이렇게 [] 안에 `.` 문자를 쓰면 여기서 `.`는 메타 문자가 아니라 '.' 문자 그대로를 의미한다. 즉, 이 정규식의 의미는 다음과 같다.

```
"a + . + b"
```

따라서 정규식 `a[.]b`는 "a.b" 문자열과 매치되고 "a0b" 문자열과는 매치되지 않는다. 혼동하지 않도록 주의하자.

`*` 문자

다음 정규식을 살펴보자.

```
ca*t
```

이 정규식은 반복을 의미하는 `*` 메타 문자를 사용했다. 여기에서 사용한 `*`은 `*` 바로 앞에 있는 문자 a가 0부터 무한대까지 반복될 수 있다는 의미이다.

> `*` *메타 문자의 반복 개수가 무한대라고 표현했는데, 실제로는 시스템 메모리, 입력 데이터 크기, 정규식 엔진의 구현 등에 의해 제한된다.*

즉, 다음과 같은 문자열이 모두 매치된다.

정규식	문자열	매치 여부	설명
`ca*t`	ct	Yes	"a"가 0번 반복되어 매치
`ca*t`	cat	Yes	"a"가 0번 이상 반복되어 매치 (1번 반복)
`ca*t`	caaat	Yes	"a"가 0번 이상 반복되어 매치 (3번 반복)

+ 문자

반복을 나타내는 또 다른 메타 문자로 `+`가 있다. `+`는 최소 1번 이상 반복될 때 사용한다. 즉, `*`가 반복 횟수 0부터라면 `+`는 반복 횟수 1부터인 것이다.

다음 정규식을 살펴보자.

```
ca+t
```

위 정규식의 의미는 다음과 같다.

```
"c + a가_1번_이상_반복 + t"
```

위 정규식에 대한 매치 여부는 다음 표와 같다.

정규식	문자열	매치 여부	설명
`ca+t`	ct	No	"a"가 0번 반복되어 매치되지 않음
`ca+t`	cat	Yes	"a"가 1번 이상 반복되어 매치 (1번 반복)
`ca+t`	caaat	Yes	"a"가 1번 이상 반복되어 매치 (3번 반복)

{} 문자와 ? 문자

반복 횟수를 3회만 또는 1회부터 3회까지만으로 제한하고 싶을 때는 어떻게 할까? 이럴 때 `{}` 메타 문자를 사용하면 반복 횟수를 고정할 수 있다. `{m, n}` 정규식을 사용하면 반복 횟수가 m부터 n까지인 문자와 매치할 수 있다. m 또는 n을 생략할 수도 있다. 만약 `{3,}` 처럼 사용하면 반복 횟수가 3 이상인 경우이고 `{,3}` 처럼 사용하면 반복 횟수가 3 이하인 경우를 의미한다. 생략된 m은 0과 동일하며, 생략된 n은 무한대의 의미를 갖는다.

> *`{1,}` 은 `+` , `{0,}` 은 `*` 와 동일하다.*

{}을 사용한 몇 가지 정규식 예시를 살펴보자.

1. {m}

```
ca{2}t
```

이 정규식의 의미는 다음과 같다.

```
"c + a를_반드시_2번_반복 + t"
```

이 정규식에 대한 매치 여부는 다음 표와 같다.

정규식	문자열	매치 여부	설명
`ca{2}t`	cat	No	"a"가 1번만 반복되어 매치되지 않음
`ca{2}t`	caat	Yes	"a"가 2번 반복되어 매치

2. {m, n}

```
ca{2,5}t
```

이 정규식의 의미는 다음과 같다.

```
"c + a를_2~5회_반복 + t"
```

이 정규식에 대한 매치 여부는 다음 표와 같다.

정규식	문자열	매치 여부	설명
`ca{2,5}t`	cat	No	"a"가 1번만 반복되어 매치되지 않음
`ca{2,5}t`	caat	Yes	"a"가 2번 반복되어 매치
`ca{2,5}t`	caaaaat	Yes	"a"가 5번 반복되어 매치

3. `?`

반복은 아니지만 이와 비슷한 기능을 하는 `?` 문자가 있다. `?` 메타 문자가 의미하는 것은 `{0, 1}` 이다.

다음 정규식을 살펴보자.

```
ab?c
```

이 정규식의 의미는 다음과 같다.

```
"a + b가_있어도_되고_없어도_됨 + c"
```

이 정규식에 대한 매치 여부는 다음 표와 같다.

정규식	문자열	매치 여부	설명
`ab?c`	abc	Yes	"b"가 1번 사용되어 매치
`ab?c`	ac	Yes	"b"가 0번 사용되어 매치

즉, b 문자가 있거나 없거나 둘 다 매치되는 경우이다.

`*`, `+`, `?` 메타 문자는 모두 `{m, n}` 형태로 고쳐 쓸 수 있지만, 이해하기 쉽고 표현도 간결한 `*`, `+`, `?` 메타 문자를 사용하는 것이 좋다.

지금까지 매우 기초적인 정규 표현식에 대해 알아보았다. 알아야 할 것들이 아직 많이 남아 있지만, 그에 앞서 파이썬으로 정규 표현식을 어떻게 사용할 수 있는지 알아보자.

파이썬에서 정규 표현식을 지원하는 re 모듈

파이썬은 정규 표현식을 지원하기 위해 re(regular expression) 모듈을 제공한다. re 모듈은 파이썬을 설치할 때 자동으로 설치되는 표준 라이브러리로, 사용 방법은 다음과 같다.

```
>>> import re
>>> p = re.compile('ab*')
```

re.compile을 사용하여 정규 표현식(위 예에서는 `ab*`)을 컴파일한다. 여기서 컴파일이란 정규식 문자열을 파이썬이 빠르게 처리할 수 있는 패턴 객체로 변환하는 과정이다. re.compile의 결과를 객체 p(컴파일된 패턴 객체)에 할당해 그 이후의 작업을 수행할 것이다.

> - *정규식을 컴파일할 때 특정 옵션을 주는 것도 가능한데, 이에 대해서는 뒤에서 자세히 살펴본다.*
> - *패턴이란 정규식을 컴파일한 결과이다.*

정규식을 이용한 문자열 검색

이제 컴파일된 패턴 객체를 사용하여 문자열 검색을 수행해 보자. 컴파일된 패턴 객체는 다음과 같은 4가지 메서드를 제공한다.

Method	목적
match()	문자열의 처음부터 정규식과 매치되는지 조사한다.
search()	문자열 전체를 검색하여 정규식과 매치되는지 조사한다.
findall()	정규식과 매치되는 모든 문자열(substring)을 리스트로 반환한다.
finditer()	정규식과 매치되는 모든 문자열(substring)을 반복 가능한 객체로 반환한다.

match, search는 정규식과 매치될 때는 match 객체를 반환하고 매치되지 않을 때는 None을 반환한다. match 객체란 정규식의 검색 결과로 반환된 객체를 말한다.

먼저 다음과 같은 패턴을 만들어 보자.

```
>>> import re
>>> p = re.compile('[a-z]+')
```

이제 이 패턴 객체로 앞에 나온 메서드를 사용하는 간단한 예를 살펴보자.

match

match 메서드는 문자열의 처음부터 정규식과 매치되는지 조사한다. 즉, 패턴이 문자열의 시작 위치(인덱스 0)에서 바로 매치되어야 한다. 앞 패턴에 match 메서드를 수행해 보자.

```
>>> m = p.match("python")
>>> print(m)
<re.Match object; span=(0, 6), match='python'>
```

"python" 문자열은 `[a-z]+` 정규식에 부합되므로 match 객체가 반환된다.

```
>>> m = p.match("3 python")
>>> print(m)
None
```

"3 python" 문자열은 처음에 나오는 문자 3이 정규식 `[a-z]+` 에 부합되지 않으므로 None이 반환된다.

match의 결과로 match 객체 또는 None을 반환하기 때문에 파이썬 정규식 프로그램은 보통 다음과 같은 흐름으로 작성한다.

```
p = re.compile(정규표현식)
m = p.match('조사할 문자열')
if m:
    print('Match found: ', m.group())
else:
    print('No match')
```

즉, match의 결괏값이 있을 때만 그다음 작업을 수행하겠다는 것이다.

search

컴파일된 패턴 객체 p를 가지고 이번에는 search 메서드를 수행해 보자.

```
>>> m = p.search("python")
>>> print(m)
<re.Match object; span=(0, 6), match='python'>
```

"python" 문자열에 search 메서드를 수행하면 match 메서드를 수행했을 때와 동일하게 매치된다.

```
>>> m = p.search("3 python")
>>> print(m)
<re.Match object; span=(2, 8), match='python'>
```

"3 python" 문자열의 첫 번째 문자는 "3"이지만, search는 문자열의 처음부터 검색하는 것이 아니라 문자열 전체를 검색하기 때문에 "3" 이후의 "python" 문자열과 매치된다.

이렇듯 match 메서드와 search 메서드는 문자열의 처음부터 검색할지 여부에 따라 다르게 사용해야 한다. 정리하면 다음과 같다.

- `p.match("3 python")` → `None` (처음부터 매치해야 하므로 "3"에서 실패)
- `p.search("3 python")` → `match='python'` (문자열 전체를 검색하므로 "python"을 찾음)

같은 문자열이라도 match는 처음부터 일치해야 하고, search는 어디에 있든 찾아낸다는 차이를 기억하자.

findall

이번에는 findall 메서드를 수행해 보자.

```
>>> result = p.findall("life is too short")
>>> print(result)
['life', 'is', 'too', 'short']
```

findall은 패턴(`[a-z]+`)과 매치되는 모든 값을 찾아 리스트로 반환한다.

finditer

이번에는 finditer 메서드를 수행해 보자.

```
>>> result = p.finditer("life is too short")
>>> print(result)
<callable_iterator object at 0x01F5E390>
>>> for r in result:
...     print(r)
...
<re.Match object; span=(0, 4), match='life'>
<re.Match object; span=(5, 7), match='is'>
<re.Match object; span=(8, 11), match='too'>
<re.Match object; span=(12, 17), match='short'>
```

finditer는 findall과 동일하지만, 그 결과로 반복 가능한 객체(iterator object)를 반환한다. 그리고 반복 가능한 객체가 포함하는 각각의 요소는 match 객체이다.

match 객체의 메서드

match 객체란 앞에서 살펴본 p.match, p.search 또는 p.finditer 메서드에 의해 반환된 매치 객체(Match Object)를 의미한다. 이제 이 match 객체에 대해서 자세히 알아보자.

앞에서 정규식을 사용한 문자열 검색을 수행하면서 아마도 다음과 같은 궁금증이 생겼을 것이다.

- 어떤 문자열이 매치되었는가?
- 매치된 문자열의 인덱스는 어디서부터 어디까지인가?

match 객체의 메서드를 사용하면 이 같은 궁금증을 해결할 수 있다. 다음 표를 살펴보자.

method	목적
group()	매치된 문자열을 반환한다.
start()	매치된 문자열의 시작 위치를 반환한다.
end()	매치된 문자열의 끝 위치를 반환한다.
span()	매치된 문자열의 (시작, 끝)에 해당하는 튜플을 반환한다.

다음 예로 확인해 보자.

```
>>> m = p.match("python")
>>> m.group()
'python'
>>> m.start()
0
>>> m.end()
6
>>> m.span()
(0, 6)
```

예상한 대로 결괏값이 출력되는 것을 확인할 수 있다. match 메서드를 수행한 결과로 반환된 match 객체의 start 메서드는 항상 0을 반환한다. match 메서드는 항상 문자열의 시작부터 조사하기 때문이다.

만약 search 메서드를 사용했다면 `m.start()` 값은 다음과 같이 다르게 나올 것이다.

```
>>> m = p.search("3 python")
>>> m.group()
'python'
>>> m.start()
2
>>> m.end()
8
>>> m.span()
(2, 8)
```

점프 투 파이썬

모듈 단위로 수행하기

지금까지 우리는 re.compile을 사용하여 컴파일된 패턴 객체로 그 이후의 작업을 수행했다. re 모듈은 이를 더 축약한 형태로 사용할 수 있는 방법을 제공한다. 다음 예를 살펴보자.

```
>>> p = re.compile('[a-z]+')
>>> m = p.match("python")
```

이 코드가 축약된 형태는 다음과 같다.

```
>>> m = re.match('[a-z]+', "python")
```

이렇게 사용하면 컴파일과 match 메서드를 한 번에 수행할 수 있다. 보통 한 번 만든 패턴 객체를 여러 번 사용해야 할 때는 이 방법보다 re.compile을 사용하는 것이 편리하다.

컴파일 옵션

정규식을 컴파일할 때 다음 옵션을 사용할 수 있다.

- DOTALL(S) - `.` (dot)이 줄바꿈 문자를 포함해 모든 문자와 매치될 수 있게 한다.
- IGNORECASE(I) - 대소문자에 관계없이 매치될 수 있게 한다.
- MULTILINE(M) - 여러 줄과 매치될 수 있게 한다. `^`, `$` 메타 문자 사용과 관계 있는 옵션이다.
- VERBOSE(X) - verbose 모드를 사용할 수 있게 한다. 정규식을 보기 편하게 만들 수 있고 주석 등을 사용할 수 있게 된다.

옵션을 사용할 때는 `re.DOTALL` 처럼 전체 옵션 이름을 써도 되고 `re.S` 처럼 약어를 써도 된다.

DOTALL, S

`.` 메타 문자는 줄바꿈 문자(`\n`)를 제외한 모든 문자와 매치되는 규칙이 있다. 만약 `\n` 문자도 포함하여 매치하고 싶다면 `re.DOTALL` 또는 `re.S` 옵션을 사용해 정규식을 컴파일하면 된다.

다음 예를 보자.

```
>>> import re
>>> p = re.compile('a.b')
>>> m = p.match('a\nb')
>>> print(m)
None
```

정규식이 `a.b` 인 경우 문자열 `a\nb` 는 매치되지 않는다는 것을 알 수 있다. `\n` 은 `.` 메타 문자와 매치되지 않기 때문이다. `\n` 문자와도 매치되게 하려면 다음과 같이 `re.DOTALL` 옵션을 사용해야 한다.

```
>>> p = re.compile('a.b', re.DOTALL)
>>> m = p.match('a\nb')
>>> print(m)
<re.Match object; span=(0, 3), match='a\nb'>
```

보통 `re.DOTALL` 옵션은 여러 줄로 이루어진 문자열에서 줄바꿈 문자에 상관없이 검색할 때 많이 사용한다.

IGNORECASE, I

`re.IGNORECASE` 또는 `re.I` 옵션은 대소문자 구별 없이 매치를 수행할 때 사용하는 옵션이다. 다음 예제를 살펴보자.

```
>>> p = re.compile('[a-z]+', re.I)
>>> p.match('python')
<re.Match object; span=(0, 6), match='python'>
>>> p.match('Python')
<re.Match object; span=(0, 6), match='Python'>
>>> p.match('PYTHON')
<re.Match object; span=(0, 6), match='PYTHON'>
```

`[a-z]+` 정규식은 소문자만을 의미하지만, `re.I` 옵션으로 대소문자 구별 없이 매치된다.

MULTILINE, M

`re.MULTILINE` 또는 `re.M` 옵션은 조금 후에 설명할 메타 문자인 `^`, `$`와 연관된 옵션이다. 이 메타 문자에 대해 간단히 설명하면 `^`는 문자열의 처음, `$`는 문자열의 마지막을 의미한다. 예를 들어 정규식이 `^python`인 경우, 문자열의 처음은 항상 python으로 시작해야 매치되고 만약 정규식이 `python$` 이라면 문자열의 마지막은 항상 python으로 끝나야 매치된다는 의미이다.

다음 예를 살펴보자.

```
# multiline.py
import re
p = re.compile("^python\s\w+")

data = """python one
life is too short
python two
you need python
python three"""

print(p.findall(data))
```

정규식 `^python\s\w+`은 python이라는 문자열로 시작하고 그 뒤에 화이트스페이스, 그 뒤에 단어가 와야 한다는 의미이다.

이 스크립트를 실행하면 다음과 같은 결과를 반환한다.

```
['python one']
```

`^` 메타 문자에 의해 python이라는 문자열을 사용한 첫 번째 줄만 매치된 것이다.

하지만 `^` 메타 문자를 문자열 전체의 처음이 아니라 각 라인의 처음으로 인식시키고 싶은 경우도 있을 것이다. 이럴 때 사용할 수 있는 옵션이 바로 `re.MULTILINE` 또는 `re.M`이다. 앞 코드를 다음과 같이 수정해 보자.

```
# multiline.py
import re
p = re.compile("^python\s\w+", [[MARK]]re.MULTILINE[[/MARK]])

data = """python one
life is too short
python two
you need python
python three"""

print(p.findall(data))
```

`re.MULTILINE` 옵션으로 인해 `^` 메타 문자가 문자열 전체가 아닌 각 줄의 처음이라는 의미를 가지게 되었다. 이 스크립트를 실행하면 다음과 같은 결과가 출력된다.

```
['python one', 'python two', 'python three']
```

즉, `re.MULTILINE` 옵션은 `^`, `$` 메타 문자를 문자열의 각 줄마다 적용해 주는 것이다.

VERBOSE, X

지금껏 알아본 정규식은 매우 간단하지만, 정규식 전문가들이 만든 정규식을 보면 거의 암호 수준이다. 정규식을 이해하려면 하나하나 조심스럽게 뜯어 봐야만 한다. 이렇게 이해하기 어려운 정규식을 주석 또는 줄 단위로 구분할 수 있다면 얼마나 보기 좋고 이해하기 쉬울까? 이 경우에는 `re.VERBOSE` 또는 `re.X` 옵션을 사용하면 된다.

다음 이메일 주소를 검증하는 정규식 예를 보자.

```
email_pattern = re.compile(r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$')
```

이 정규식이 한눈에 이해되는가? 무엇을 하는 건지 알기 어렵다. 이제 `re.VERBOSE` 옵션을 사용한 같은 정규식을 살펴보자.

```
email_pattern = re.compile(r"""
    ^                       # 문자열의 시작
    [a-zA-Z0-9._%+-]+       # 사용자명: 영문자, 숫자, 특수문자
    @                       # @ 기호
    [a-zA-Z0-9.-]+          # 도메인명: 영문자, 숫자, 점, 하이픈
    \.                      # 점(.)
    [a-zA-Z]{2,}            # 최상위 도메인: 영문자 2자 이상
    $                       # 문자열의 끝
""", [[MARK]]re.VERBOSE[[/MARK]])
```

첫 번째와 두 번째 예를 비교해 보면 컴파일된 패턴 객체는 모두 동일한 역할을 한다. 하지만 정규식이 복잡할 경우, 두 번째처럼 주석을 적고 여러 줄로 표현하는 것이 훨씬 가독성이 좋다는 것을 알 수 있다.

`re.VERBOSE` 옵션을 사용하면 문자열에 사용된 주석과 화이트스페이스는 컴파일할 때 제거된다(단, [ ] 안에 사용한 화이트스페이스는 제외).

역슬래시 문제

정규 표현식을 파이썬에서 사용할 때 혼란을 주는 요소가 1가지 있는데, 바로 역슬래시(`\`)이다.

예를 들어 어떤 파일 안에 있는 `"\section"` 문자열을 찾기 위한 정규식을 만든다고 가정해 보자.

```
\section
```

이 정규식은 `\s` 문자가 whitespace로 해석되어 의도한 대로 매치가 이루어지지 않는다.

이 표현은 다음과 동일한 의미이다.

```
[ \t\n\r\f\v]ection
```

의도한 대로 매치하고 싶다면 다음과 같이 변경해야 한다.

```
\\section
```

즉, 앞 정규식에서 사용한 `\` 문자가 문자열 자체라는 것을 알려 주기 위해 역슬래시 2개를 사용해 이스케이프 처리를 해야 한다.

따라서 위 정규식을 컴파일하려면 다음과 같이 작성해야 한다.

```
>>> p = re.compile('\\section')
```

그런데 여기에서 또 하나의 문제가 발견된다. 이처럼 정규식을 만들어서 컴파일하면 실제 파이썬 정규식 엔진에는 파이썬 문자열 리터럴 규칙에 따라 `\\`이 `\`로 변경되어 `\section`이 전달된다.

> *이 문제는 이와 같은 정규식을 파이썬에서 사용할 때만 발생한다(파이썬의 리터럴 규칙). 유닉스의 grep, vi 등에서는 이러한 문제가 없다.*

결국 정규식 엔진에 `\\` 문자를 전달하려면 파이썬은 `\\\\`처럼 역슬래시를 4개나 사용해야 한다.

> *정규식 엔진은 정규식을 해석하고 수행하는 모듈이다.*

```
>>> p = re.compile('\\\\section')
```

이렇게 해야만 원하는 결과를 얻을 수 있다. 하지만 너무 복잡하지 않은가?

만약 이와 같이 `\`를 사용한 표현이 계속 반복되는 정규식이라면 너무 복잡해서 이해하기 어려울 것이다. 이 문제를 해결하려면 raw string 표현법을 사용해야 한다. 그 방법은 다음과 같다.

```
>>> p = re.compile(r'\\section')
```

이와 같이 정규식 문자열 앞에 r 문자를 삽입하면 이 정규식은 raw string 규칙에 의해 역슬래시 2개 대신 1개만 써도 2개를 쓴 것과 동일한 의미를 가지게 된다.

> *만약 역슬래시를 사용하지 않는 정규식이라면 r의 유무에 상관없이 동일한 정규식이 될 것이다.*

이 변환 과정을 표로 정리하면 다음과 같다.

파이썬 코드	파이썬 문자열 변환	정규식 엔진에 전달
`'\\section'`	`\section`	`\s` 로 해석되어 의도와 다르게 동작
`'\\\\section'`	`\\section`	`\section` 을 문자 그대로 검색 (정상)
`r'\\section'`	`\\section`	`\section` 을 문자 그대로 검색 (정상)

08-3 강력한 정규 표현식의 세계로

이제 08-2에서 배우지 않은 몇몇 메타 문자의 의미를 살펴보고 그룹을 만드는 법, 전방 탐색 등 더욱 강력한 정규 표현식에 대해서 살펴보자.

- 문자열 소비가 없는 메타 문자
  - |
  - ^
  - $ $메타 문자는^메타 문자와 반대의 경우이다. 즉,$ 는 문자열의 끝과 매치한다는 것을 의미한다.
  - \A
  - \Z
  - \b
  - \B
- 그루핑
  - 여러 문자를 하나로 묶어서 반복 처리하기
  - 매치된 문자열에서 원하는 부분만 추출하기
  - 그루핑된 문자열 재참조하기
  - 그루핑된 문자열에 이름 붙이기
- 전방 탐색
  - 긍정형 전방 탐색
  - 부정형 전방 탐색
- 문자열 바꾸기
  - sub 메서드 사용 시 참조 구문 사용하기
  - sub 메서드의 매개변수로 함수 넣기
- greedy와 non-greedy

문자열 소비가 없는 메타 문자

아직 살펴보지 않은 메타 문자들을 알아보자. 이번에 다룰 메타 문자들은 앞에서 배운 것들과 특별한 차이점이 있다. 지금까지 배운 `+`, `*`, `[]`, `{}` 같은 메타 문자들은 매치되면 해당 문자들을 "사용해버린다"고 생각하면 된다. 이를 좀 더 쉽게 표현하면, 문자열을 읽는 위치(커서)가 매치된 만큼 앞으로 이동한다는 뜻이다. 예를 들어 `aac` 라는 문자열에서 `a+` 패턴을 찾으면 `aa` 부분이 매치되고, 읽는 위치가 `aa` 뒤로 이동하여 그다음부터는 남은 `c` 에서 다시 탐색을 시작한다. 하지만 이번에 배울 메타 문자들은 조건만 확인하고 실제로는 읽는 위치를 이동시키지 않는 특별한 성질을 가지고 있다.

|

`|` 메타 문자는 or과 동일한 의미로 사용된다. `A|B`라는 정규식이 있다면 A 또는 B라는 의미가 된다.

```
>>> p = re.compile('Crow|Servo')
>>> m = p.match('CrowHello')
>>> print(m)
<re.Match object; span=(0, 4), match='Crow'>
```

^

`^` 메타 문자는 문자열의 맨 처음과 일치한다는 것을 의미한다. 앞에서 살펴본 컴파일 옵션 `re.MULTILINE`을 사용할 경우에는 여러 줄의 문자열일 때 각 줄의 처음과 일치하게 된다.

다음 예를 살펴보자.

```
>>> print(re.search('^Life', 'Life is too short'))
<re.Match object; span=(0, 4), match='Life'>
>>> print(re.search('^Life', 'My Life'))
None
```

`^Life` 정규식은 Life 문자열이 처음에 온 경우에는 매치하지만, 처음 위치가 아닌 경우에는 매치되지 않음을 알 수 있다.

$ `$ 메타 문자는 ^ 메타 문자와 반대의 경우이다. 즉, $`는 문자열의 끝과 매치한다는 것을 의미한다.

다음 예를 살펴보자.

```
>>> print(re.search('short$', 'Life is too short'))
<re.Match object; span=(12, 17), match='short'>
>>> print(re.search('short$', 'Life is too short, you need python'))
None
```

`short$` 정규식은 검색할 문자열이 short로 끝난 경우에는 매치되지만, 그렇지 않은 경우에는 매치되지 않음을 알 수 있다. > `^` 또는 `$` 문자를 메타 문자가 아닌 문자 그 자체로 매치하고 싶은 경우에는 `\^`, `\$`로 작성하면 된다.

\A

\A 는 문자열의 처음과 매치된다는 것을 의미한다. ^ 메타 문자와 동일한 의미이지만, re.MULTILINE 옵션을 사용할 경우에는 다르게 해석된다. re.MULTILINE 옵션을 사용할 경우 ^ 은 각 줄의 문자열 처음과 매치되지만, \A 는 줄과 상관없이 전체 문자열의 처음하고만 매치된다.

\Z

\Z 는 문자열의 끝과 매치된다는 것을 의미한다. 이것 역시 \A 와 동일하게 re.MULTILINE 옵션을 사용할 경우, $ 메타 문자와는 달리 전체 문자열의 끝과 매치된다.

\b

\b 는 단어 구분자(word boundary)이다. 보통 단어는 화이트스페이스에 의해 구분된다.

다음 예를 살펴보자.

```
>>> p = re.compile(r'\bclass\b')
>>> print(p.search('no class at all'))
<re.Match object; span=(3, 8), match='class'>
```

\bclass\b 정규식은 앞뒤가 화이트스페이스로 구분된 class라는 단어와 매치된다는 것을 의미한다. 따라서 no class at all의 class라는 단어와 매치됨을 확인할 수 있다.

```
>>> print(p.search('the declassified algorithm'))
None
```

위 예의 the declassified algorithm 문자열 안에도 class 문자열이 포함되어 있기는 하지만, 화이트스페이스로 구분된 단어가 아니므로 매치되지 않는다.

```
>>> print(p.search('one subclass is'))
None
```

subclass 문자열 역시 class 앞에 sub 문자열이 더해져 있으므로 매치되지 않음을 알 수 있다.

\b 메타 문자를 사용할 때 주의해야 할 점이 있다. \b 는 파이썬 리터럴 규칙에 따르면 백스페이스(backspace)를 의미하므로 백스페이스가 아닌 단어 구분자라는 것을 알려 주기 위해 r'\bclass\b' 처럼 raw string임을 알려 주는 r 을 반드시 붙여야 한다.

\B

\B 메타 문자는 \b 메타 문자와 반대의 경우이다. 즉, 화이트스페이스로 구분된 단어가 아닌 경우에만 매치된다.

```
>>> p = re.compile(r'\Bclass\B')
>>> print(p.search('no class at all'))
None
>>> print(p.search('the declassified algorithm'))
<re.Match object; span=(6, 11), match='class'>
>>> print(p.search('one subclass is'))
None
```

\Bclass\B 는 class의 앞과 뒤 모두 단어 경계가 아니어야 매치된다. "declassified"에서는 class 앞에 "de", 뒤에 "ified"가 붙어 있으므로 양쪽 모두 단어 경계가 아니어서 매치된다. 반면 "subclass"에서는 class 뒤에 화이트스페이스(단어 경계)가 있으므로 매치되지 않는다.

그루핑

그루핑(grouping)은 정규 표현식에서 매우 중요한 개념이다. 그루핑을 사용하는 목적은 크게 두 가지다. 첫째는 여러 문자를 하나로 묶어서 반복 처리하는 것이고, 둘째는 매치된 문자열에서 원하는 부분만 추출하는 것이다.

여러 문자를 하나로 묶어서 반복 처리하기

예를 들어 ABC 문자열이 계속해서 반복되는지 조사하고 싶다고 하자. 이럴 때 괄호 () 를 사용해서 여러 문자를 하나로 묶을 수 있다.

```
(ABC)+
```

이렇게 하면 ABC라는 세 글자가 하나의 단위로 취급되어, ABC가 한 번 이상 반복되는 패턴을 찾을 수 있다.

```
>>> p = re.compile('(ABC)+')
>>> m = p.search('ABCABCABC OK?')
>>> print(m)
<re.Match object; span=(0, 9), match='ABCABCABC'>
>>> print(m.group())
ABCABCABC
```

매치된 문자열에서 원하는 부분만 추출하기

그루핑의 더 중요한 용도는 매치된 문자열 중에서 특정 부분만 뽑아내는 것이다. 다음 예를 통해 알아보자.

```
>>> p = re.compile(r"\w+\s+\d+[-]\d+[-]\d+")
>>> m = p.search("park 010-1234-1234")
```

위 정규식 `\w+\s+\d+[-]\d+[-]\d+`은 `이름 + 공백 + 전화번호` 형태의 문자열을 찾는다. 하지만 전체 문자열이 아니라 이름 부분만 따로 추출하고 싶다면 어떻게 해야 할까? 바로 그룹을 사용하면 된다.

```
>>> p = re.compile(r"[[MARK]](\w+)[[/MARK]]\s+\d+[-]\d+[-]\d+")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group(1))
park
```

이름에 해당하는 `\w+` 부분을 `(\w+)`과 같이 그룹으로 만들면 match 객체의 group(인덱스) 메서드를 사용하여 그루핑된 부분의 문자열만 뽑아낼 수 있다. group 메서드의 인덱스는 다음과 같은 의미를 가진다.

group(인덱스)	설명
group(0)	매치된 전체 문자열 (group()과 동일)
group(1)	첫 번째 그룹에 해당하는 문자열
group(2)	두 번째 그룹에 해당하는 문자열
group(n)	n 번째 그룹에 해당하는 문자열

> *group()처럼 인수 없이 호출하면 group(0)과 동일하게 매치된 전체 문자열을 반환한다.*

다음 예제를 계속해서 살펴보자.

```
>>> p = re.compile(r"(\w+)\s+[[MARK]](\d+[-]\d+[-]\d+)[[/MARK]]")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group(2))
010-1234-1234
```

이번에는 전화번호 부분을 그룹 `(\d+[-]\d+[-]\d+)`로 만들었다. 이렇게 하면 `group(2)`처럼 사용해 전화번호만 뽑아낼 수 있다.

만약 전화번호 중에서 국번만 뽑아내고 싶으면 어떻게 해야 할까? 다음과 같이 국번 부분을 또 그루핑하면 된다.

```
>>> p = re.compile(r"(\w+)\s+([[MARK]](\d+)[[/MARK]][-]\d+[-]\d+)")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group(3))
010
```

위 예에서 볼 수 있듯이 `(\w+)\s+((\d+)[-]\d+[-]\d+)`처럼 그룹을 중첩해 사용할 수도 있다. 그룹이 중첩된 경우는 바깥쪽부터 시작해 안쪽으로 들어갈수록 인덱스 값이 증가한다.

그루핑된 문자열 재참조하기

그룹의 또 하나 좋은 점은 한 번 그루핑한 문자열을 재참조(backreferences)할 수 있다는 점이다. 다음 예를 살펴보자.

```
>>> p = re.compile(r'(\b\w+)\s+\1')
>>> p.search('Paris in the the spring').group()
'the the'
```

정규식 `(\b\w+)\s+\1`은 `(그룹) + " " + 그룹과 동일한 단어`와 매치된다는 것을 의미한다. 이렇게 정규식을 만들면 2개의 동일한 단어를 연속적으로 사용해야만 매치된다. 이를 가능하게 하는 것이 바로 재참조 메타 문자인 `\1`이다. `\1`은 정규식의 그룹 중 첫 번째 그룹을 가리킨다.

위 예제의 매치 과정을 단계별로 살펴보면 다음과 같다.

1. `(\b\w+)`가 "Paris"와 매치되어 그룹 1에 "Paris"가 저장된다.
2. `\s+`가 공백과 매치된다.
3. `\1`이 그룹 1의 값인 "Paris"를 참조하여 다음 단어와 비교하지만, "in"이므로 매치 실패한다.
4. 같은 방식으로 "in", "the"를 차례로 시도한다.
5. `(\b\w+)`가 "the"와 매치되어 그룹 1에 "the"가 저장되고, `\s+` 뒤의 `\1`이 다음 단어 "the"와 일치하므로 최종 매치가 성공한다.

> *두 번째 그룹을 참조하려면 `\2` 를 사용하면 된다.*

그루핑된 문자열에 이름 붙이기

정규식 안에 그룹이 무척 많아진다고 가정해 보자. 예를 들어 정규식 안에 그룹이 10개 이상만 되어도 매우 혼란스러울 것이다. 게다가 정규식이 수정되면서 그룹이 추가, 삭제되면 그 그룹을 인덱스로 참조한 프로그램도 모두 변경해 주어야 하는 위험도 갖게 된다.

만약 그룹을 인덱스가 아닌 이름(named groups)으로 참조할 수 있다면 어떨까? 그렇다면 이런 문제에서 해방되지 않을까?

이러한 이유로 정규식은 그룹을 만들 때 그룹 이름을 지정할 수 있게 했다. 그 방법은 다음과 같다.

```
(?P<name>\w+)\s+((\d+)[-]\d+[-]\d+)
```

이 정규식은 앞에서 본 이름과 전화번호를 추출하는 정규식이다. 기존과 달라진 부분은 다음과 같다.

```
(\w+) → (?P<name>\w+)
```

대단히 복잡해진 것처럼 보이지만, `(\w+)` 라는 그룹에 name이라는 이름을 붙인 것에 불과하다. 여기에서 사용한 `(?...)` 표현식은 정규 표현식의 확장 구문이다. 이 확장 구문을 사용하면 가독성이 떨어지는 대신 강력한 기능을 사용할 수 있다.

그룹에 이름을 지어 주려면 다음과 같은 확장 구문을 사용해야 한다.

```
(?P<그룹명>...)
```

그룹에 이름을 지정하고 참조하는 다음 예를 살펴보자.

```
>>> p = re.compile(r"(?P<name>\w+)\s+((\d+)[-]\d+[-]\d+)")
>>> m = p.search("park 010-1234-1234")
>>> print(m.group("name"))
park
```

위 예에서 볼 수 있듯이 name이라는 그룹 이름으로 참조할 수 있다.

그룹 이름을 사용하면 정규식 안에서 재참조하는 것도 가능하다.

```
>>> p = re.compile(r'(?P<word>\b\w+)\s+[[MARK]](?P=word)[[/MARK]]')
>>> p.search('Paris in the the spring').group()
'the the'
```

위 예에서 볼 수 있듯이 재참조할 때는 `(?P=그룹이름)`이라는 확장 구문을 사용해야 한다.

전방 탐색

전방 탐색은 정규식에서 가장 이해하기 어려운 개념 중 하나다. 하지만 한 번 이해하면 매우 유용한 기능이므로 차근차근 알아보자.

전방 탐색이 무엇인지 간단한 예를 통해 먼저 알아보자.

```
>>> p = re.compile(".+:")
>>> m = p.search("http://google.com")
>>> print(m.group())
http:
```

위 예에서 정규식 `.+:`는 "어떤 문자든 1개 이상 + 콜론"을 의미한다. 그래서 결과로 `http:`가 나온다. 그런데 만약 콜론(`:`)이 있는지는 확인하고 싶지만, 실제 결과에서는 콜론을 제외하고 `http`만 얻고 싶다면 어떻게 해야 할까?

바로 이런 상황에서 전방 탐색을 사용한다. 전방 탐색은 "앞을 내다보기"라고 생각하면 된다. 즉, 특정 조건이 앞에 있는지 확인만 하고, 실제로는 그 부분을 결과에 포함시키지 않는다.

전방 탐색에는 두 가지 종류가 있다.

- 긍정형 전방 탐색(`(?=...)`): "뒤에 `...`가 오는지 확인하되, 실제로는 포함하지 않기"
- 부정형 전방 탐색(`(?!...)`): "뒤에 `...`가 오지 않는지 확인하되, 실제로는 포함하지 않기"

긍정형 전방 탐색

긍정형 전방 탐색을 사용하면 위 예제의 문제를 해결할 수 있다. 콜론(`:`)이 있는지 확인하되 결과에서는 제외하는 방법을 살펴보자.

```
>>> p = re.compile(".+(?=:)")
>>> m = p.search("http://google.com")
>>> print(m.group())
http
```

기존 정규식 `.+:`에서 `:`를 `(?=:)`로 바꾸었다. 이렇게 하면 콜론이 뒤에 있는지 확인하지만, 실제 매치 결과에는 콜론을 포함하지 않는다. 쉽게 말해서 "콜론이 뒤에 오는 문자들을 찾되, 콜론은 가져오지 마"라는 뜻이다.

부정형 전방 탐색

부정형 전방 탐색은 긍정형과 반대다. "뒤에 특정 내용이 오지 않는 경우"를 찾을 때 사용한다. 좀 더 복잡한 예제를 통해 알아보자.

다음 정규식은 파일 이름을 찾는 패턴이다.

```
.*[.].*$
```

이 정규식은 foo.bar, autoexec.bat, sendmail.cf 같은 파일 이름과 매치된다. 그런데 여기서 ".bat 파일은 제외하고 싶다"는 조건이 생겼다고 해보자.

가장 먼저 생각할 수 있는 방법은 다음과 같다.

```
.*[.][^b].*$
```

"확장자가 b로 시작하지 않는 파일"이라는 뜻이다. 하지만 이 방법은 foo.bar 같은 파일도 함께 제외해 버린다. bar의 첫 글자가 b이기 때문이다.

그래서 좀 더 정확하게 만들어보자.

```
.*[.]([^b]..|.[^a].|..[^t])$
```

이 정규식은 "확장자의 첫 번째 문자가 b가 아니거나, 두 번째 문자가 a가 아니거나, 세 번째 문자가 t가 아닌 경우"를 의미한다. 이렇게 하면 foo.bar는 제외되지 않고 autoexec.bat은 제외된다. 하지만 sendmail.cf 같이 확장자가 2글자인 파일은 매치되지 않는 문제가 생긴다.

이 문제까지 해결하려면 다음과 같이 더 복잡해진다.

```
.*[.]([^b].?.?|.[^a]?.?|..?[^t]?)$
```

이제 확장자가 2글자인 파일도 처리된다. 하지만 정규식이 매우 복잡해졌다. 여기에 .exe 파일도 제외하라는 조건이 추가된다면 어떻게 될까?

이런 상황에서 부정형 전방 탐색이 해결책이 된다.

```
.*[.][[MARK]](?!bat$)[[/MARK]].*$
```

`(?!bat$)`는 "뒤에 bat가 오지 않는 경우"를 의미한다. 즉, 확장자가 bat가 아닌 파일만 찾겠다는 뜻이다.

여기서 exe 파일도 함께 제외하고 싶다면 다음과 같이 쓸 수 있다.

```
.*[.](?!bat$|exe$).*$
```

이렇게 하면 ".bat 파일도 .exe 파일도 아닌 파일"만 찾을 수 있다. 복잡한 조건도 간단하게 표현할 수 있는 것이 부정형 전방 탐색의 장점이다.

문자열 바꾸기

sub 메서드를 사용하면 정규식과 매치되는 부분을 다른 문자로 쉽게 바꿀 수 있다. 다음 예를 살펴보자.

```
>>> p = re.compile('(blue|white|red)')
>>> p.sub('colour', 'blue socks and red shoes')
'colour socks and colour shoes'
```

sub 메서드의 첫 번째 인수는 "바꿀 문자열(replacement)"이 되고, 두 번째 인수는 "대상 문자열"이 된다. 이 예에서 볼 수 있듯이 blue 또는 white 또는 red라는 문자열이 colour라는 문자열로 바뀌는 것을 확인할 수 있다.

그런데 딱 한 번만 바꾸고 싶은 경우도 있다. 바꾸기 횟수를 제어하려면 다음과 같이 세 번째 인수에 count 값을 설정하면 된다.

```
>>> p.sub('colour', 'blue socks and red shoes', [[MARK]]count=1[[/MARK]])
'colour socks and red shoes'
```

처음 일치하는 blue만 colour라는 문자열로 한 번만 바뀌는 것을 알 수 있다.

점프 투 파이썬

sub 메서드와 유사한 subn 메서드

subn 역시 sub와 동일한 기능을 하지만, 반환 결과를 튜플로 돌려준다는 차이가 있다. 반환된 튜플의 첫 번째 요소는 변경된 문자열, 두 번째 요소는 바꾸기가 발생한 횟수이다.

```
>>> p = re.compile('(blue|white|red)')
>>> p.subn( 'colour', 'blue socks and red shoes')
('colour socks and colour shoes', 2)
```

sub 메서드 사용 시 참조 구문 사용하기

sub 메서드를 사용할 때 참조 구문을 사용할 수 있다. 다음 예를 살펴보자.

```
>>> p = re.compile(r"(?P<name>\w+)\s+(?P<phone>(\d+)[-]\d+[-]\d+)")
>>> print(p.sub("\g<phone> \g<name>", "park 010-1234-1234"))
010-1234-1234 park
```

이 예는 `이름 + 전화번호`의 문자열을 `전화번호 + 이름`으로 바꾼다. sub의 바꿀 문자열 부분에 `\g<그룹_이름>`을 사용하면 정규식의 그룹 이름을 참조할 수 있다.

다음과 같이 그룹 이름 대신 참조 번호를 사용해도 마찬가지 결과를 돌려준다.

```
>>> p = re.compile(r"(?P<name>\w+)\s+(?P<phone>(\d+)[-]\d+[-]\d+)")
>>> print(p.sub("\g<2> \g<1>", "park 010-1234-1234"))
010-1234-1234 park
```

sub 메서드의 매개변수로 함수 넣기

sub 메서드의 첫 번째 인수에 함수를 전달할 수도 있다. 다음 예를 살펴보자.

```
>>> def hexrepl(match):
...     value = int(match.group())
...     return hex(value)
...
>>> p = re.compile(r'\d+')
>>> p.sub(hexrepl, 'Call 65490 for printing, 49152 for user code.')
'Call 0xffd2 for printing, 0xc000 for user code.'
```

hexrepl은 match 객체를 입력으로 받아 16진수로 변환하여 돌려주는 함수이다. sub의 첫 번째 인수로 함수를 사용할 경우, 해당 함수의 첫 번째 매개변수에는 정규식과 매치된 match 객체가 입력된다. 그리고 매치되는 문자열은 함수의 반환값으로 바뀌게 된다. 위 예제의 동작 흐름을 정리하면 다음과 같다.

1. `\d+` 패턴이 "65490"과 매치 → hexrepl 함수에 match 객체 전달 → `hex(65490)` 반환 → "0xffd2"로 교체
2. `\d+` 패턴이 "49152"와 매치 → hexrepl 함수에 match 객체 전달 → `hex(49152)` 반환 → "0xc000"으로 교체

이처럼 매치될 때마다 함수가 반복 호출되어 각 매치를 개별적으로 처리할 수 있다.

greedy와 non-greedy

정규식에서는 '탐욕스러운(greedy)'이라는 표현을 종종 쓴다. 이 말은 어떤 의미일까? 다음 예제를 살펴보자.

```
>>> s = '<html><head><title>Title</title>'
>>> len(s)
32
>>> print(re.match('<.*>', s).span())
(0, 32)
>>> print(re.match('<.*>', s).group())
<html><head><title>Title</title>
```

`<.*>` 정규식의 매치 결과로 `<html>` 문자열을 반환해 주기를 기대했을 것이다. 하지만 `*` 메타 문자는 매우 탐욕스러워서 매치할 수 있는 최대한의 문자열인 `<html><head><title>Title</title>` 문자열을 모두 소비해 버렸다. 어떻게 하면 이 탐욕스러움을 제한하고 `<html>` 문자열까지만 소비하도록 막을 수 있을까?

다음과 같이 non-greedy 문자인 `?`를 사용하면 `*`의 탐욕을 제한할 수 있다.

```
>>> print(re.match('<.*?>', s).group())
<html>
```

non-greedy 문자인 `?` 는 `*?`, `+?`, `??`, `{m,n}?` 와 같이 사용할 수 있다. 되도록 최소한으로 반복을 수행하도록 도와주는 역할을 한다.

> *08-2에서 배운 `?` 는 단독으로 사용되어 "0회 또는 1회 반복"을 의미하지만, 여기서의 `?` 는 `*`, `+` 등 반복 메타 문자 뒤에 붙어 "최소한으로 반복하라"는 의미로 사용된다. 같은 기호이지만 위치에 따라 역할이 다르므로 혼동하지 않도록 주의하자.*